Лабораторная работа №5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

Марданов Самир, группа 11-401

Постановка и реализация. Обучение проксимальным градиентным методом: за одну итерацию за один проход по строкам X считаются прогноз $\hat{y}_i$, ошибка e_i и частичные суммы g_j, g_b; затем $w_j \leftarrow S_{\eta\lambda}(w_j - \eta g_j)$, $b \leftarrow b - \eta g_b$. Варианты: `serial`; `simd` — `#pragma omp simd reduction(+:s)` для суммы по признакам в прогнозе и `omp simd` для накопления градиента и обновления w; `omp` — одна параллельная область на все итерации, строки делятся `omp for schedule(static)`, каждый поток копит градиент в своём буфере (длина выровнена до 64 байт против false sharing), после `barrier` буферы суммируются `omp for` по j — гонок нет; `omp_simd` — то же плюс SIMD-директивы. Данные: $x_{ij} \sim U[-1, 1]$ (признаки уже центрированы и одного масштаба, нормализация не нужна), истинные веса ненулевые у каждого 4-го признака, $y = Xw^* + 0.5 +$ шум U[−0.1, 0.1], генератор splitmix64, seed = 42. Начально $w = 0$, $b = 0$; $\eta = 0.5$, $K = 100$ (функция потерь сходится: при $K = 100$ и 300 совпадает до 12 знаков), $\lambda = 10^{-3}$, $\varepsilon = 10^{-6}$. Генерация, выделение памяти, вычисление F и вывод вне замера (`omp_get_wtime()`), в таблицах медиана 5–7 запусков. Компьютер: Apple M1 Pro (6 производительных + 2 энергоэффективных ядра, 16 ГБ, NEON 128 бит), macOS, GCC 16.2: `gcc-16 -O3 -march=native -fopenmp linreg_simd.c -o linreg_simd -lm`, запуск `./linreg_simd -n N -d d -k K -l λ -e η -t p -r reps`; по умолчанию 8 потоков.

3.1. Корректность ($N = 10^4$, $d = 32$, допуск 10^{-9})

Реализация	F(w, b)	Δ_w	OK?
Последовательная	$1.111643513218 \cdot 10^{-2}$	0	да
OpenMP	$1.111643513218 \cdot 10^{-2}$	0	да
SIMD	$1.111643513218 \cdot 10^{-2}$	0	да
OpenMP + SIMD	$1.111643513218 \cdot 10^{-2}$	0	да

Прочие запуски: $\Delta_w \le 2.4 \cdot 10^{-15}$

3.2. $N = 10^5$, $d = 32$, $K = 100$, 8 потоков

Вариант	Время, с	S
Последовательная	0.3329	1.00
OpenMP	0.0840	3.96
SIMD	0.2243	1.48
OpenMP + SIMD	0.0622	5.35

3.7. Флаги ($N = 10^5$, $d = 32$, 8 потоков)

Флаги	Время, с	S	Примечание
`-O0`	1.5767	1.00	serial, без векторизации
`-O3`	0.3354	4.70	serial, вектор. градиент
`-O3 -march=native`	0.3323	4.74	serial, NEON 128 бит
… + `omp simd`	0.2272	6.94	simd; omp_simd 0.065 с

3.3. Влияние N ($d = 32$), время, с

N	serial	OpenMP	SIMD	OMP+SIMD
10^4	0.0332	0.0169	0.0225	0.0146
10^5	0.3329	0.0840	0.2243	0.0622
10^6	3.3285	0.7275	2.2431	0.5143

S: OpenMP 1.96 / 3.96 / 4.58; OMP+SIMD 2.27 / 5.35 / 6.47

3.4. Влияние d ($N = 10^5$), время, с

d	serial	OpenMP	SIMD	OMP+SIMD	T_{ser}/T_{simd}
8	0.0718	0.0249	0.0722	0.0270	0.99
32	0.3323	0.0827	0.2239	0.0627	1.48
128	1.7609	0.3552	0.9855	0.2263	1.79
256	3.6092	0.8311	2.0430	0.4862	1.77

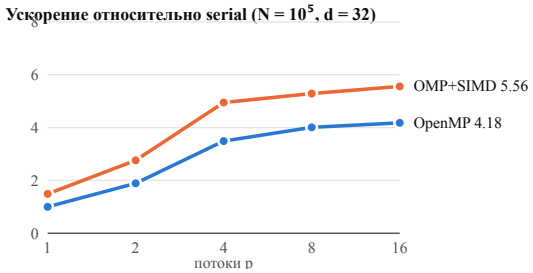
3.6. Влияние λ ($N = 10^5$, $d = 32$)

λ	F(w, b)	$\lvert w_j\rvert < \varepsilon$	Время, с
0	$1.6706 \cdot 10^{-3}$	0	0.0617
10^{-4}	$2.6194 \cdot 10^{-3}$	13	0.0627
10^{-3}	$1.1144 \cdot 10^{-2}$	24	0.0641
10^{-2}	$9.5316 \cdot 10^{-2}$	24	0.0624
10^{-1}	$8.2979 \cdot 10^{-1}$	24	0.0634

время — OpenMP + SIMD, 8 потоков

3.5. Влияние числа потоков ($N = 10^5$, $d = 32$)

p	OpenMP, с	S_p	E_p	OMP+SIMD, с	S_p	E_p
1	0.3332	1.00	1.00	0.2238	1.00	1.00
2	0.1762	1.89	0.95	0.1207	1.85	0.93
4	0.0953	3.49	0.87	0.0673	3.33	0.83
8	0.0831	4.01	0.50	0.0629	3.56	0.44
16	0.0796	4.18	0.26	0.0599	3.73	0.23



Векторизация (`-fopt-info-vec`). Векторизованы автоматически (V2DF, 16 байт, unroll 2): накопление градиента `g[j] += e*x[j]` в serial и omp (с версионированием цикла «because of possible aliasing», т.к. буфер g может пересекаться с X) и обновление w через $S_\alpha(z) = z - \min(\max(z, -\alpha), \alpha)$ (вариант с ветвлениями давал «unsupported control flow»). Не векторизована сумма прогноза `s += x[j]*w[j]` («Loop costings may not be worthwhile»): без `-ffast-math` GCC не вправе переставлять сложения double и строит лишь невыгодную упорядоченную редукцию. Также не векторизованы внешний цикл по строкам (два вложенных цикла) и суммирование буферов потоков (короткий цикл по t с ветвлением по j). `omp simd reduction(+:s)` разрешает перестановку — прогноз векторизуется; во всех трёх циклах SIMD-варианта проверки алиасинга исчезают. Проверка: с `-ffast-math` serial без директив ускоряется до 0.196 с, т.е. весь выигрыш SIMD — именно от редукции.

Выводы и ответы на вопросы 3.8. 1. Самая быстрая — OpenMP + SIMD на всех N и $d \ge 32$ (до 6.5× при $N = 10^6$ и 7.8× при $d = 128$). При $d = 8$ SIMD ничего не даёт (вектор длины 8 — 4 пары NEON, накладные расходы редукции съедают выигрыш), и OpenMP без SIMD чуть быстрее. С ростом N ускорение OpenMP растёт (1.96 → 4.58): при $N = 10^4$ итерация длится ≈ 0.15 мс и два барьера плюс суммирование буферов составляют заметную долю. Выигрыш SIMD растёт с d (0.99 → 1.79) и насыщается к $d = 128$: длинные внутренние циклы лучше амортизируют горизонтальное сложение редукции и хвосты. **2.** OpenMP + SIMD быстрее OpenMP в 1.15–1.7 раза (при $d \ge 32$), но его S_p относительно собственной однопоточной версии ниже (3.56 против 4.01 на 8 потоках): векторизованный код быстрее упирается в пропускную способность памяти ($N = 10^5$, $d = 32$ — 25.6 МБ на итерацию, больше кэша), а доля непараллельных частей (барьеры, редукция буферов) в меньшем времени больше. **3.** До 4 потоков $E_p \ge 0.83$, на 8 падает до 0.44–0.50: 2 из 8 ядер энергоэффективные и медленные, при `schedule(static)` все ждут их на барьере; 16 потоков (больше ядер) дают лишь +4–5% — насыщение. **4.** См. абзац выше: автоматически — градиент и обновление w, после `omp simd` — также редукция прогноза; `-march=native` на M1 не меняет ширину векторов (SVE нет), поэтому почти не влияет; основной эффект даёт `-O3` (4.7×). **5.** С ростом λ коэффициенты сжимаются к нулю на ≈ 3λ (гессиан ≈ $I/3$): при $\lambda = 10^{-4}$ обнуляются 13 из 24 «шумовых» весов, начиная с 10^{-3} — все 24 (ровно истинный носитель), ненулевые 8 при этом уменьшаются, поэтому F растёт и за счёт MSE, и за счёт $\lambda\|w\|_1$; на одних и тех же данных при $\lambda = 0$ $F \approx \sigma^2/2 = 1.67 \cdot 10^{-3}$. Время от λ не зависит. **6.** Ограничения: доступ к памяти (каждая итерация читает всю X, вычислительная интенсивность ≈ 0.5 флопа на байт), невекторизуемая без перестановки редукция, барьеры и объединение $p \cdot d$ частичных градиентов (заметно при малых N) и неоднородные ядра. Корректность подтверждена во всех запусках: отличия только из-за порядка суммирования, $\le 2.4 \cdot 10^{-15}$ при допуске 10^{-9}.